



UP6. Servicios Web

Desarrollo Web en Entorno Servidor

Profesora: Silvia Vilar Pérez

curso 2025-2026

Contenidos

- Arquitecturas de programación orientadas a servicios (SOA)
- Tipos de Servicios
- Protocolos para los servicios
- SOAP.
- REST API
- Códigos respuesta HTTP

Arquitecturas Orientadas a Servicios (SOA)

Un servicio es un componente SW que:

- Está definido por una interfaz
- Está disponible de forma remota
- Opera sobre la capa de negocio
- Es reutilizable

Así pues, una Arquitectura Orientada a Servicios se basa en dichos servicios para obtener o realizar las tareas necesarias de forma análoga a la Programación Orientada a Objetos que usa los Objetos para sus fines

Tipos de Servicios

Un servicio puede clasificarse como:

- De **entidad**: Realizan operaciones CRUD (Create, Read, Update, Delete) sobre las entidades de negocio. Ejemplo: cliente, factura, empleado, etc.
- **Funcional**: Proporciona una funcionalidad reutilizable y centralizada en la que otros servicios puedan confiar, debe ser lo más autónomo posible. Ejemplo: pasarela de pago
- De **proceso**: Representa un conjunto de tareas relacionadas que forman parte del mismo proceso atómico. Puede llamar a otros servicios para dichas tareas. Ejemplo: un servicio para procesar la contratación de un nuevo empleado usaría los servicios: dar de alta al empleado, solicitar una nueva estación de trabajo al departamento de IT, enviar un correo electrónico informativo al departamento al que se asigna el empleado, etc.

Protocolos para Servicios Web

Los servicios Web se basan en unos protocolos (entendidos como normas o reglas para la comunicación y el funcionamiento) de entre los que destacamos dos:

- **SOAP** (Simple Object Access Protocol) basado en XML-RCP (XML Remote Procedure Call): Genera un XML con tres partes: <Envelope> (elemento raíz), <Header> (subelemento opcional para pasar información a la aplicación) y <Body> (subelemento con los datos, puede incluir errores con la etiqueta <Fault>). Este XML se envía mediante HTTP
- **REST** (Representational State Transfer): Proporciona una API basada en los métodos del protocolo HTTP (GET, POST, PUT y DELETE) para realizar diferentes operaciones entre el cliente y la aplicación que ofrece el servicio web y utiliza los códigos de respuesta de HTTP

Simple Object Access Protocol (SOAP)

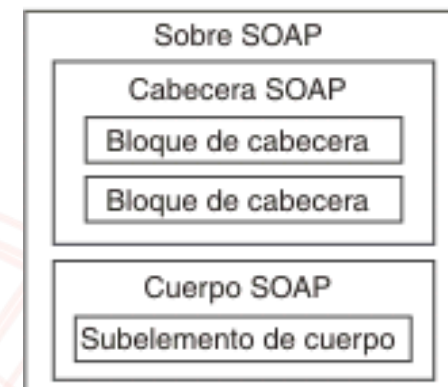
Creada en 1998 es mantenida por el World Wide Web Consortium (W3C)

- El Cliente Web Service realiza una petición que es atendida por el servicio Servidor Web Service o Service Provider quien está continuamente a la escucha de nuevas peticiones
- La comunicación se realiza utilizando XML para transmitir los datos
- El servidor, indica los datos de conexión y el catálogo de servicios en un fichero con extensión .WSDL

Simple Object Access Protocol (SOAP)

Un mensaje SOAP es un documento XML con los siguientes elementos:

- Un sobre (envelope) que identifica el documento XML como mensaje SOAP
- Una cabecera (header) que contiene la información de la cabecera
- Un cuerpo (body) que contienen la información de la petición (call) y respuesta (response)
- Un elemento (fault) para información de errores e información de estado



```
POST /InStock HTTP/1.1
Host: www.example.org
Content-Type: application/soap+xml; charset=utf-8
Content-Length: nnn

<?xml version="1.0"?>

<soap:Envelope
xmlns:soap="http://www.w3.org/2003/05/soap-envelope/"
soap:encodingStyle="http://www.w3.org/2003/05/soap-encoding">

<soap:Body xmlns:m="http://www.example.org/stock">
  <m:GetStockPrice>
    <m:StockName>IBM</m:StockName>
  </m:GetStockPrice>
</soap:Body>

</soap:Envelope>
```

```
HTTP/1.1 200 OK
Content-Type: application/soap+xml; charset=utf-8
Content-Length: nnn

<?xml version="1.0"?>

<soap:Envelope
xmlns:soap="http://www.w3.org/2003/05/soap-envelope/"
soap:encodingStyle="http://www.w3.org/2003/05/soap-encoding">

<soap:Body xmlns:m="http://www.example.org/stock">
  <m:GetStockPriceResponse>
    <m:Price>34.5</m:Price>
  </m:GetStockPriceResponse>
</soap:Body>

</soap:Envelope>
```

Representational State Transfer (REST)

- REST es un sistema o interfaz que utiliza el protocolo HTTP para realizar operaciones CRUD (Create, Read, Update y Delete) usando formatos de datos específicos como XML o JSON.
- El uso de REST posibilita que se use una URI (Identificador Uniforme de Recurso) para las operaciones CRUD.
- Se usará una uri pero la operación CRUD a realizar se basará en los métodos GET, POST, PUT y DELETE con el que se enviará la petición HTTP

Correspondencia CRUD-REST

- Los servicios REST usan los métodos HTTP (POST, GET, PUT y DELETE)
- Los objetos del Modelo implementan los métodos index, store, show, update y destroy

SQL	CRUD	REST	URI
SELECT from students	students.index	GET	api/students
INSERT into students	students.store	POST	api/students
SELECT from students where id	students.show	GET	api/students/{id}
UPDATE students where id	students.update	PUT	api/students/{id}
DELETE from student where id	students.destroy	DELETE	api/students/{id}

La URI se genera con el nombre del Modelo (en este caso la clase Students) tras la url de nuestra API. En el caso de acceder a un student concreto, se indica el número del ítem tras la /

Ejemplo students.json para API REST

```
[  
  {  
    "id": 1,  
    "name": "Silvia"  
  },  
  {  
    "id": 2,  
    "name": "Martin"  
  },  
  {  
    "id": 3,  
    "name": "Javi"  
  },  
  {  
    "id": 4,  
    "name": "Angel"  
  },  
  {  
    "id": 5,  
    "name": "Manuel Maria"  
  }  
]
```

Ejemplo creación API REST

<?php

//Indicamos el nombre del JSON con los estudiantes

\$file="students.json";

// Cargamos el JSON students

\$students =json_decode(file_get_contents(\$file),true);

// Detectar método HTTP en el servidor

\$method = \$_SERVER['REQUEST_METHOD'];

// Detectar recurso e ID /students/3 → resource=students, id=3

\$uri = explode('/', trim(\$_SERVER['REQUEST_URI'], '/'));

\$resource = isset(\$uri[0]) ? \$uri[0] : null;

\$id = isset(\$uri[1]) ? \$uri[1] : null;

// Solo gestionamos /students

if (\$resource !== 'students') {

http_response_code(404);

echo json_encode(["error" => "Resource not found"]);

exit;

}

// (Continúa en las siguientes diapositivas con la gestión de cada método)

Ejemplo creación API REST - GET

```
/**
 * MÉTODO GET → INDEX Y SHOW
 */
if ($method === 'GET') {

    // GET /students → INDEX (mostrar todos los estudiantes)
    if (!$id) {
        http_response_code(200);
        echo json_encode($students);
        exit;
    }

    // GET /students/{id} → SHOW (mostrar un estudiante)
    foreach ($students as $student) {
        if ($student["id"] == $id) {
            http_response_code(200);
            echo json_encode($student);
            exit;
        }
    }

    // Si no se encuentra el estudiante
    http_response_code(404);
    echo json_encode(["error" => "Student not found"]);
    exit;
}
```

Ejemplo creación API REST - POST

```
/**
 * MÉTODO POST → STORE
 * */
if ($method === 'POST') {

    // Leemos el JSON enviado por el cliente
    // php://input permite leer el cuerpo de la petición
    $raw = file_get_contents("php://input");
    $input = json_decode($raw, true); // true para obtener un array asociativo

    // Calcular nuevo ID
    // Obtenemos el último elemento del array y le sumamos 1
    // Los IDs pueden no ser consecutivos, así que buscamos el máximo
    $ids = array_column($students, 'id'); // Extraemos los IDs
    $newId = max($ids) + 1; // Calculamos el nuevo ID

    $newStudent = [
        "id" => $newId,
        "name" => isset($input["name"]) ? $input["name"] : "Unnamed"
    ];
    $students[] = $newStudent;
    file_put_contents($file, json_encode($students, JSON_PRETTY_PRINT)); //actualiza fichero

    http_response_code(201); // Created
    echo json_encode($newStudent);
    exit;
}
```


Ejemplo creación API REST - PUT

```
/**
 * MÉTODO PUT → UPDATE
 * */
if ($method === 'PUT') {
    // Leemos el JSON enviado por el cliente
    $input = json_decode(file_get_contents("php://input"), true);
    // Buscamos el estudiante
    foreach ($students as $index => $student) {
        if ($student["id"] == $id) {
            // Actualizamos solo el campo name
            $updatedName = isset($input["name"]) ? $input["name"] : $student["name"];
            $students[$index]["name"] = $updatedName;
            // guardamos el cambio en el fichero
            file_put_contents($file, json_encode($students, JSON_PRETTY_PRINT));

            http_response_code(200);
            echo json_encode($updatedStudent);
            exit;
        }
    }
    // Si no existe el ID
    http_response_code(404);
    echo json_encode(["error" => "Student not found"]);
    exit;
}
```

Ejemplo creación API REST - DELETE

```
/**
 * MÉTODO DELETE → DESTROY
 */
if ($method === 'DELETE') {

    foreach ($students as $index => $student) {
        if ($student["id"] == $id) {
            unset($students[$index]); //Eliminamos el student
            file_put_contents($file, json_encode($students,JSON_PRETTY_PRINT));
            http_response_code(204); // No Content
            exit;
        }
    }
    // Si no existe el ID
    http_response_code(404);
    echo json_encode(["error" => "Student not found"]);
    exit;
}

?>
```

Ejemplo ejecucion API REST

Endpoint **GET /students**

Status: 200

Reason: OK

Body: [{"id": 1, "name": "Silvia"}, {"id": 2, "name": "Martín"}, {"id": 3, "name": "Javi"}, {"id": 4, "name": "Angel"}, {"id": 5, "name": "Manuel Maria"}]

Endpoint **GET /students/1**

Status: 200

Reason: OK

Body: {"id": 1, "name": "Silvia"}

Endpoint **POST /students**

Status: 201

Reason: Created

Body: {"id": 6, "name": "Vanesa"}

Endpoint **PUT /students/6**

Status: 200

Reason: OK

Body: {"id": 6, "name": "Vanesa Updated"}

Endpoint **DELETE /students/6**

Status: 204

Reason: No Content

Body:

Códigos de Respuesta HTTP

- Códigos **1XX**: representan información sobre una petición normalmente incompleta, (cuando la petición es muy larga, se envía antes una cabecera para comprobar si se puede procesar dicha petición).
- Códigos **2xx**: representan peticiones que se han podido atender satisfactoriamente. (**200** - respuesta estándar para peticiones correctas, **201** - se ha insertado o creado un nuevo recurso en el servidor o **204** – [No content] la petición se ha atendido pero no se devuelve nada como respuesta).
- Códigos **3xx**: son códigos de redirección a otro recurso del servidor. (**301** - el recurso solicitado se ha movido permanentemente a otra URL, **304** - el recurso solicitado no ha cambiado desde la última vez que se solicitó).
- Códigos **4xx**: indican un error por parte del cliente. (**404** – se solicita una URL o recurso que no existe, **401** - cliente no autorizado, **400** - los datos de la petición no son correctos).
- Códigos **5xx**: indican un error por parte del servidor. (**500** - error interno del servidor, **504** - error de timeout por tiempo excesivo en emitir la respuesta).